DATA 442/642 · Advanced Machine Learning · Week 13

13.1 Back-Propagation Challenges

Vanishing gradients and other training failure modes.

These notes walk through the lecture slides. The original deck is unchanged; use **(slides)** on the course hub if you want that PDF.

Figures and notes follow Géron, *Hands-On Machine Learning*; Deisenroth, Faisal, and Ong, *Mathematics for Machine Learning*; and Theodoridis.

1. Backpropagation, in one idea

Gradient descent, with an efficient gradient. Two passes (one forward, one backward) give the derivative of the error with respect to every weight and every bias. Tweak each connection the way the gradient says. Repeat until the net converges.

---

2. The algorithm

Work in **mini-batches** (e.g. 32 rows). A full pass over the training set is an **epoch**.

Forward pass. The batch hits the input layer, then each hidden layer in turn, then the output layer. Same computation as prediction, except you **keep every intermediate** z and a: the backward pass needs them.

Error. A **loss** compares desired output to actual output.

Backward pass. Chain rule: how much each output connection contributed to the error; then the layer below; then the layer below that, down to the inputs. The error gradient is pushed **backward** through the net (the name).

Step. Ordinary gradient descent on all weights, using those gradients.

---

3. Forward, compactly

Input x, weights W, biases b, activations σ. Set $a \leftarrow x$. For each layer l:

$$z = W^{(l)}a + b^{(l)}, \qquad a = \sigma(z)\,.$$

Return the last a (the prediction).

---

4. Backward, compactly

Run the forward pass first. Start from $\delta = (\partial L \,/\, \partial a)\,\sigma'(z)$ at the output. Walking from last layer to first:

$dW^{(l)} = \delta\,(a^{(l-1)})^{\top}, \qquad db^{(l)} = \sum \delta,$

then

$\delta \leftarrow \left((W^{(l)})^{\top}\delta\right) \odot \sigma'(z)$

for the layer below. At the input, the same formulas with $a^{(0)} = x$. Return all dW, db.

---

5. Automatic differentiation

Automatic differentiation (AD) splits a map into elementary ops whose derivatives you know, then applies the chain rule.

Mode	Chain rule runs	Efficient when
Forward	inputs → outputs	few inputs, many outputs
Reverse	outputs → inputs	many inputs, few outputs

Neural nets have many parameters and a scalar loss. **Reverse mode** is backpropagation. That is why SGD and its variants can see a gradient with respect to a huge θ.

---

6. Challenges

Problem	What it is
Local minima	A hole that is not the global min. Descent can sit there.
Plateaus, saddles, flats	$\nabla J \approx 0$ over a wide set. Slow progress. In high dimension this is common near the optimum: flat versus truly done is hard to tell.
Cliffs and exploding gradients	A small $\Delta\theta$ makes a huge ΔJ. Gradients blow up; steps become unstable. Deep nets, **RNNs**, and deep **CNNs** see this when gradients accumulate.
Long-term dependencies	A prediction now depends on an input far in the past. RNNs struggle because gradients **vanish** or explode. Language modeling and translation need distant context.
Inexact gradients	Numerical error, approximations, or distributed-training glitches. Tiny errors in a numerical derivative add up.
Local vs global structure	A neighborhood that looks promising does not lead to the global min. Nonconvex J is full of holes, saddles, and flats that mislead a local method.

Practice

1. Name one reason a ReLU helps with vanishing gradients.
2. Neural nets have many parameters and one loss. Which AD mode is backpropagation, and why is the other a poor fit?

3. Name one optimization obstacle from this lecture that is *not* "the gradient is almost zero."